

Security Engineering

Flask Tutorial 1

Contents

1	Introduction	1
2	Installation	1
3	Flask Features	3
4	Thoughts Application: Functionality	9

1 Introduction

In this tutorial, we learn how to build web applications in Python using Flask.

Python already has libraries for servlet-style web programming. However, writing such application is often tedious and time consuming. Flask provides developers with functionalities that simplify many aspects of web programming at the price of following some minor conventions.

Here we specifically focus on Flask concepts of routes, endpoints, canonical URLs, HTTP methods, requests, sessions, and templates. In the next tutorial we will also learn how to manage persistent state and user credentials.

As a running example for this tutorial, we implement Thoughts application: a simple web application which allows *Persons* to create *Thoughts*, share them with others, and vote for their favorite thoughts.

2 Installation

Docker Download and extract the Flask Docker file.

You can create your application file `<app_name>.py` in the `app` folder. To run a server with a Flask application for the first time type:

```
export PROJECT_NAME=<app_name>
docker-compose up --build
```

If you visit `http://127.0.0.1:5000` you can see your application running.

To stop the server use `docker-compose down`. You can omit the `--build` flag next time you want to run the server.

Local Alternatively, you can install Python on your machine. Flask supports Python version 3.7 and newer. Consult Python documentation for installing it on your particular system.

Once you have Python, create a project folder and its virtual environment:

```
mkdir <project_name>
cd <project_name>
python3 -m venv .venv
```

activate the virtual environment with:

```
. .venv/bin/activate
```

or, if you are using Windows, with:

```
.venv/Scripts/activate.bat
```

Your shell prompt will change to indicate that a virtual environment is active. Now all dependencies you install will only apply to the current project and will not affect any other projects you may have. (To deactivate the virtual environment just type `deactivate`.)

Next, while `venv` is active install Flask:

```
pip install Flask
```

this will install Flask and all its dependencies.

To run the application, use

```
flask --app app run
```

The `--app` option is used to select the `.py` file where the Flask application is. If you visit `http://127.0.0.1:5000` you can see your application running.

3 Flask Features

In this section we briefly overview basic features of Flask using a minimal example¹.

We provide this minimal example in a python file `app.py`, where you can uncomment individual parts (marked with enumerated steps) and run the application.

Basic app Suppose you created `mini-app` folder and inside `app.py` file with

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def main():
    return "Hello world!"
```

The code first imports the Flask class. We will assume that this import statement and the subsequent initialization of `app` precede all subsequent code snippets here. An instance of the Flask class `app` represents a web application. The first argument (`__name__`, which holds the name of the `.py` file) is the name of the application's main module or package. Based on this Flask establishes the root folder of the project and then looks for other resources (such as templates and static files). Flask instance provides decorators, like `route(URL)` which tells Flask to treat requests to URL as calls to function `main`.

The function returns the message we want to display in the user's browser. The default content type is HTML, so any HTML in the string will be rendered by the browser. If you inspect the page in our example, there is no HTML, as we just return the string `Hello world`.

Multiple routes You can use the `route(URL)` decorator multiple times to map URLs to functions. See the example below:

```
@app.route("/")
def main():
    return "Main menu"

@app.route("/hello")
def hello():
    return "Hello world!"
```

¹For more information visit <https://flask.palletsprojects.com/en/2.2.x/>

The URL parameter is called the canonical URL for the decorated function (also called an endpoint). If the canonical URL has a trailing slash (/), then when accessing the URL without a trailing slash, Flask would redirect to the canonical URL. The opposite is not true: if the canonical URL does not have a trailing slash, accessing it with a trailing slash produces 404 "Not Found" error.

Parameters and converter types You can specify parts of a canonical URL to be treated as its endpoint's parameter by adding `< ... >` around it. The text between the delimiters is the endpoint parameter's name, i.e., if `<variable_name>` is in the URL the endpoint has `variable_name` parameter. See the example below:

```
@app.route("/")
def main():
    return "Main menu"

@app.route("/hello/<name>")
def hello(name="world"):
    return f"Hello {name}!"

@app.route("/hello/<int:number>")
def bye(number=0):
    return f"Hello int {number}!"
```

Optionally, you can use a converter type to specify the type of the argument like `<converter:variable_name>`. Other converter types are `string`, `int`, `float`, `path`, `uuid`

Redirect and building URLs Conceptually, a redirect calls an endpoint programmatically. Difference to actual function call is that the user observes the change to the visited URL. Redirection is done using Flask's `redirect` object and passing the target endpoint's canonical URL. See the example below:

```
from flask import redirect, url_for

@app.route("/")
def main():
    return "Main menu"

@app.route("/hello/<name>")
def hello(name="world"):
    return f"Hello {name}!"

@app.route("/hello/<int:number>")
def bye(number=0):
```

```

if number == 0:
    return redirect(url_for('hello', name="zero"))
print(url_for('main'))
print(url_for('hello', name="John Doe"))
print(url_for('bye', number=1))
return f"Hello int {number}!"

```

As you can see instead of providing the canonical URL manually, we use the `url_for()` function. It accepts the *name of the endpoint* as its first argument and any number of named parameters, each corresponding to a variable part of the URL rule. Unknown variable parts are appended to the URL as query parameters.

HTTP methods By default, an endpoint decorated with `route` only answers to GET requests. You can use the `methods` argument of the route decorator to set acceptable HTTP methods. Data sent by the user can be accessed using the global `request` object. The object is actually a proxy to request objects that are context-local, and hence thread-safe. Consider the following example:

```

from flask import request

@app.route("/")
def main():
    return '''<form action='/hello' method='post'>
        <input type='text' name='name'>
        <input type="submit" value="Submit">
    </form>'''

@app.route("/hello", methods=['GET', 'POST'])
def hello():
    if request.method == 'POST':
        return "POST! " + request.form["name"]
    else:
        return "GET! " + request.args["name"]

@app.get('/login')
def login_get():
    return "show_the_login_form"

@app.post('/login')
def login_post():
    return "do_the_login"

```

The `hello` endpoint accepts both GET and POST methods. The current method is available via the `method` attribute of the `request` object. To access POST (or PUT) data use the `form` attribute. To access GET data use the `args` attribute. You can also separate an endpoint into two based

on the method they accept by replacing the `route` decorator with `get` and `post` decorators (see the last two endpoints above).

Sessions The session object is a key-value map that allows storing per-user information. This is implemented using cookies: if a user's browser provides a previously obtained cookie, the session object will contain the key-value bindings previously set for that user. Flask uses secret key to sign the cookies. This means is that the user could look at the contents of a cookie but cannot forge it without the secret key. Consider an example:

```
from flask import session, request, redirect, url_for

app.secret_key = b'...'

@app.route('/')
def main():
    if 'username' in session:
        return f'Logged in as {session["username"]}'
    return 'You are not logged in'

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        session['username'] = request.form['username']
        return redirect(url_for('main'))
    return '''<form method="post">
        <p><input type="text" name="username">
        <p><input type="submit" value="Login">
    </form>'''

@app.route('/logout')
def logout():
    session.pop('username', None)
    return redirect(url_for('main'))
```

First, the secret key needs to be set (we use a hardcoded key for the sake of simplicity). The `main` endpoint tests whether there is a value bound to the `username` key. This would be the case if a (returning) user made a request and presented a cookie. Otherwise, a user needs to visit the `login` endpoint using a GET request. This returns a simple login form with only a username field. Once the user sends a POST request with a selected username it is saved in the session object. The session object automatically sends a cookie at that point. The `logout` endpoint just clears the `username` binding in the `session` object, which then unsets the cookie.

Templates So far all pages returned by the endpoints consisted of only text. One could extend the strings returned by each endpoint to be complete

HTML pages. This would be repetitive and tedious. Because of that Flask uses the Jinja template engine, that we briefly present here².

The main idea of a template is to specify a family of files by separately specifying the static parts (i.e., those that do not change across the file family) only once, and then specifying the variable parts (i.e., those that change across the file family) individually. Combining the variable parts and the static part is done by a template engine, which defines a particular syntax for template specification. We now introduce Jinja's syntax by example. Consider the following `root.html` file:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>App</title>
</head>
<body>
  <h1>
    <a href="/">
      Minimal example
    </a>
  </h1>
  {% block content %}{% endblock %}
</body>
</html>
```

It is reasonable to specify all the HTML boilerplate once. Here we do a bit more: we also add a heading with the name of the application that can take the user to the canonical URL /. The red part is Jinja syntax for defining the variable part of the page (called a block). Suppose that we have the main `main.html` page and the `auth.html` page to login. Both can define the content block in their own way:

```
{% extends 'root.html' %}

{% block content %}

<label>Username:</label><div> {{ username }} </div>
<a href="/logout">Logout</a>

{% endblock %}
```

The main page defines a label `username`, a `div` element, and a logout link. The `div` element contains a Jinja expression of the form `{{e}}`. Jinja evaluates `e` to a string and replaces the expression with it. Here it is a variable

²Full documentation: <https://jinja.palletsprojects.com/en/3.1.x/templates/>

username containing the current user's name. Finally the `auth.html` replaces the content block with a login form:

```
{% extends 'root.html' %}

{% block content %}
<form action="/login" method="post">
  <label>Username</label>
  <input type="text" name="username">
  <label>Password</label>
  <input type="password" name="password">
  <button type="submit">Login</button>
</form>
{% if wrong_pass %}
  <span>Incorrect username/password pair</span>
{% endif %}
{% endblock %}
```

This template also shows Jinja's `if` construct. The page additionally shows feedback to the user if `wrong_pass` variable evaluates to true. Jinja also has a `for` construct: `{{for i in seq}} {{endfor}}`. We now show a minimal Flask application that uses these templates:

```
from flask import render_template, session, request, redirect, url_for

app.secret_key = b'...'

@app.route('/')
def main():
    if 'username' in session:
        return render_template('main.html', username=session['username'])
    return render_template('auth.html', wrong_pass=False)

credentials = {
    "admin" : "admin",
    "alice" : "secret",
    "bob"   : "69DRcmFc1vFJN1T"
}

def check(username, password):
    return username in credentials and password == credentials[username]

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        if check(request.form['username'], request.form['password']):
            session['username'] = request.form['username']
            return redirect(url_for('main'))
        else:
            return render_template('auth.html', wrong_pass=True)
    return render_template('auth.html')

@app.route('/logout')
def logout():
    session.pop('username', None)
```



```
return redirect(url_for('main'))
```

As you can see Flask uses the `render_template` function to activate Jinja and create an HTML page to send to the user. The function additionally takes a variable number of named parameters. These depend on the template being rendered. For example, for `main.html` we set the `username` parameter, while for `auth.html` we set `wrong_pass`. Flask assumes that all template files are located in the `templates` folder in the root project directory.

This last application also combines all the Flask features we've seen so far. Feel free to start the application and use it to see how these features behave. Note that the user credentials are mismanaged for simplicity.

4 Thoughts Application: Functionality

In what follows, we introduce a simple application for Thoughts: a simple web application which allows *Persons* to create *Thoughts*, share them with others, and vote for their favorite thoughts.

Now you are tasked with implementing the Thoughts application using Python and Flask. In this first tutorial, no security policies are required to be implemented: users can create thoughts, and vote for any thought arbitrary many times.

We provide a sketch of the solution in this tutorial and a complete solution next week (including some security policies implemented). Since you haven't yet learned how to persist application state in a database, we focus on a version of the application that keeps its state in memory only. In the next tutorial we will improve this application further.

Let's start by creating a new virtual environment and installing Flask there. The data model of the application, consists of two classes and an enum:

```
from enum import Enum, auto

class Role(Enum):
    USER = auto()
    SYSTEM = auto()

class Person:
    def __init__(self, name, password, myRole):
        self._name = name
        self._password = password
        self._myRole = myRole
        self._created = set()
        self._voted = []
```

```

class Thought:
    ids = 0
    def __init__(self, content):
        self._id = Thought.ids
        Thought.ids = Thought.ids+1
        self._content = content
        self._createdBy = None
        self._votedBy = []

```

Recall that `created` association was one-to-many with set semantics, while the `voted` association was many-to-many with bag (multiset) semantics. Here we use lists to represent multisets. Make sure you create a getter and setters for all fields, as well as `add` and `remove` functions for the association ends with appropriate multiplicities as so:

```

class Person:
    # ...
    def addcreated(self, t):
        self._created.add(t)
        t._createdBy = self

    def removecreated(self, t):
        self._created.remove(t)
        del t._createdBy

    def addvoted(self, t):
        self._voted.append(t)
        t._votedBy.append(self)

    def removevoted(self, t):
        self._voted.remove(t)
        t._votedBy.remove(self)

class Thought:
    # ...
    def addvotedBy(self, p):
        self._votedBy.append(p)
        p._voted.append(self)

    def removevotedBy(self, p):
        self._votedBy.remove(p)
        p._voted.remove(self)

```

Now our application's initial state is just:

```

persons = set()
thoughts = []

```

Now let's move to routes and templates. Templates can be similar as in our last example, except `main.html` needs to define a table of thoughts, and buttons for publishing the thoughts, voting for a thought and deleting a thought. As for routes a good starting point is to define one for each action a user can do:

```
@app.route('/')
def index():
    pass

@app.route('/login', methods=['POST'])
def login():
    pass

@app.route('/logout', methods=['POST'])
def logout():
    pass

@app.route('/register', methods=['POST'])
def register():
    pass

@app.route('/add_thought', methods=['POST'])
def add_thought():
    pass

@app.route('/vote_thought')
def vote_thought():
    pass

@app.route('/delete_thought')
def delete_thought():
    pass
```

Your task is to implement the business logic of these routes. We will provide you with the templates, so watch out for the variables used there.